

1 Parcours par circulation d'un jeton

Hypothèses : Graphe quelconque avec un initiateur. Canaux de communication asynchrones et fiables.

Principe : Un initiateur crée un jeton qui circule dans le graphe. Lorsque i reçoit le jeton pour la première fois, il désigne comme son père l'émetteur du jeton. Lorsque i reçoit le jeton, pour la première fois ou non, il l'envoie à un de ses voisins privé de son père et à qui il n'a pas déjà envoyé le jeton. Si i a déjà envoyé le jeton à tous ses voisins privé de son père, il l'envoie alors à son père. Finalement, on obtient un arbre enraciné en l'initiateur. Cet arbre n'a aucune propriété particulière.

Algorithm 1 - Parcours par circulation d'un jeton

———— **Connaissance** ————

$vois_i$: ensemble des voisins du site i

———— **Variables** ————

$pere_i$: identifiant de site, initialisé à $ndef$

$utilise_i$: tableau de booléen (indiqué par l'identifiant des voisins), initialisé à *Faux*

———— **Algorithme** ————

Initialement $\rightarrow$

$\triangleright$ Un seul site

$pere_i \leftarrow i$

Choisir $j_0 \in vois_i$

$utilise_i[j_0] \leftarrow Vrai$

Envoyer Jeton() à j_0

Sur réception de Jeton() de $j_0 \rightarrow$

if $pere_i = ndef$ **then**

$pere_i \leftarrow j_0$

if $\forall j \in vois_i, utilise_i[j] = Vrai$ **then**

STOP

else

if $\exists j \in vois_i, j \neq pere_i \wedge \neg utilise_i[j]$ **then**

Choisir $j \in vois_i \setminus \{pere_i\} \wedge \neg utilise_i[j]$

$utilise_i[j] \leftarrow Vrai$

Envoyer Jeton() à j

else

$utilise_i[pere_i] \leftarrow Vrai$

Envoyer Jeton() à $pere_i$

Exercice 1. – Calcul de la taille dans un graphe identifié de topologie quelconque –

Donner un algorithme distribué utilisant un parcours et qui permet à un initiateur de déterminer la taille d'un système distribué. La taille d'un système distribué est définie comme le nombre de sites appartenant à ce système. Initialement, un site ne connaît que ses voisins immédiats.

Exercice 2. – Parcours avec plusieurs initiateurs –

On suppose ici que le système distribué est muni de plusieurs initiateurs. Donner un algorithme distribué basé sur le parcours qui permet de construire un arbre couvrant du réseau enraciné en l'un des initiateurs. L'idée est que chaque initiateur lance un parcours, en créant un jeton qu'il étiquette de son identifiant. Il y a donc plusieurs jetons qui vont parcourir le graphe et lorsque "deux parcours se rencontrent", c'est celui qui a le plus grand identifiant qui "gagne".

Exo.1

Compte le nombre de sommets

Message Jeton(v)
 $\uparrow$
 entier

Variable: compteur

Init :

compteur $\leftarrow 1$

:

2e method. Init

Envoyer Jeton (compteur++) à j_0

Envoyer Jeton (compteur) à j_0

Sur réception Jeton (v)

if $père_i = \text{ndef}$:

;

compteur $\leftarrow v + 1$

;
;
} identique.

On incrémente le compteur la dernière fois que l'on aura jeton

On incrémente le compteur la 1^{ère} fois que l'on reçoit le jeton

else

utilise $[père_i] \leftarrow \text{vrai}$

compteur ++

Envoyer Jeton (compteur) à $père_i$

identique

Exo 1. bis : le nbre d'arêtes.

Initialise une variable locale $arêtes_i = 0$

Initialement $\rightarrow$

;
arêtes $i++$

Envoyer Jeton ($arêtes_i$) à j_0 .

Reception de $v \rightarrow$

if ..
if ..
stop

else

;
arêtes $i = v + 1$

Envoyer Jeton ($arêtes_i$) à j_1

;

2 Propagation d'information avec retour

(PIF pour "Propagation of Information with Feedback")

Hypothèses : Graphe quelconque avec un initiateur. Canaux de communication asynchrones et fiables.

Principe : Il y a un initiateur qui initie le PIF. Il envoie un jeton à tous ses voisins, puis attend le jeton de tous ses voisins. Voici ce que fait un site non initiateur : à la première réception du jeton, il désigne comme son père, le site émetteur de ce jeton, puis il diffuse ce jeton à tous ses voisins privé de son père. Enfin, il attend que tous ses voisins privé de son père lui renvoient le jeton. Alors, le site transmet le jeton à son père. Cet algorithme effectue un PIF et construit en même temps un arbre couvrant.

Algorithm 2 Algorithme de PIF, local au site i

———— **Connaissance** ————

$vois_i$: ensemble des voisins du site i

———— **Variables** ————

$nbRecu_i$: nombre de messages reçus, initialisé à 0

$pere_i$: identifiant de site, initialisé à $ndef$

$init_i \in \{Vrai, Faux\}$ initialisé à Faux

———— **Algorithme** ————

Initialement $\rightarrow$ /* Sur un site exactement */

$init_i \leftarrow Vrai$

for all $j \in vois_i$ **do**

Envoyer *Jeton()* à j

Sur réception de *Jeton()* de $k \rightarrow$

$nbRecu_i ++$

if $init_i$ **then**

if $nbRecu_i = |vois_i|$ **then**

Décider ; STOP_GLOBAL

else

if $pere_i = ndef$ **then**

$pere_i \leftarrow k$

for all $j \in vois_i \setminus \{k\}$ **do**

Envoyer *Jeton()* à j

if $nbRecu_i = |vois_i|$ **then**

Envoyer *Jeton()* à $pere_i$

Exercice 3. – Election de leader dans un graphe identifié de topologie quelconque –

On souhaite écrire un algorithme distribué d'élection de leader dans un système distribué identifié, de sorte que le leader soit le site d'identifiant maximal. On suppose qu'il existe un unique initiateur dans le réseau (non nécessairement le site d'identifiant maximal).

1. En se basant sur l'algorithme du PIF ci-dessus, écrire un algorithme qui effectue cette élection et tel que, à la fin de l'exécution, l'initiateur du PIF connaisse l'identifiant du leader.
2. Ajouter le nécessaire à votre algorithme afin qu'à la fin de l'exécution, tous les sites connaissent l'identifiant du leader.

Exo.3

1) PIF

Init.

Envoyer Jeton(id) à j

Sur Reception de Jeton(v) de k

```
if  
else  
  if  
  if  
    Envoyer Jeton( $\max(Cid, v)$ ) à pèrei
```

idée : dans le PIF, on veut construire l'ensemble "fils" du chaque nœud, pour pouvoir faire la diffusion du max à partir de l'initiateur.

→ Lorsqu'un nœud remonte à la fin de son PIF l'info à son père, à la réception, le père met à jour fils.

```

corrigé ex3 q2 : ——— Initialement ——— (un seul site, l'initiateur)
initi ← Vrai
for all j ∈ voisi do
    Envoyer Jeton() à j

———— Sur réception de Jeton() de k ———
nbRecui ++
if initi then
    // Phase montante : réception d'un retour de fils ou d'arête transversale
    // (traitement dans "Sur réception de JetonRetour" ci-dessous)
    // Rien à faire ici pour l'initiateur sur Jeton()
else
    if perei = ndef then
        // Première réception : k devient le père de i
        perei ← k
        if |voisi| = 1 then
            // i est une feuille, renvoie immédiatement à son père
            Envoyer JetonRetour(idi, estFils=Vrai) à perei
        else
            // Diffuse à tous ses voisins sauf son père
            for all j ∈ voisi \ {k} do
                Envoyer Jeton() à j
    else
        // Réception depuis un voisin j ≠ perei : arête transversale
        // j a déjà un père, on lui répond pour débloquer son compteur
        Envoyer JetonRetour(idi, estFils=Faux) à k

———— Sur réception de JetonRetour(val, estFils) de j ———
nbRecui ++
maxi ← max(maxi, val)
if estFils = Vrai then
    filsi ← filsi ∪ {j}

if initi then
    if nbRecui = |voisi| then
        // Tous les retours reçus : l'initiateur connaît le leader
        leaderi ← maxi
        // Phase descendante : diffusion du résultat à tous les fils
        for all j ∈ filsi do
            Envoyer Résultat(leaderi) à j
        // L'initiateur se déclare lui-même informé
        Décider
    else
        if nbRecui = |voisi| - 1 then
            // Tous les retours reçus sauf celui du père (qui n'en envoie pas)
            // i remonte le max de son sous-arbre à son père
            Envoyer JetonRetour(maxi, estFils=Vrai) à perei

———— Sur réception de Résultat(id) de k ———
leaderi ← id
// Propagation aux fils
for all j ∈ filsi do
    Envoyer Résultat(id) à j
// Si filsi = ∅ (feuille), rien à propager

```

Hauteur de l'arbre couvrant

S'il y a un parcours de Jeton, calculer la hauteur de l'arbre couvrant construit lors du parcours

Connaissance :

$Vois_i$: ensemble des voisins du site i .

Variables

$père_i$: identifiant du père
 $prof_i \leftarrow 0$ (profondeur du noeud i)
 $h_i \leftarrow 0$ (hauteur maximale)

Variable de communication

$Jeton(d, h)$ (d : hauteur de l'émetteur, h : hauteur max)

Initialement (sur l'initiateur i)

$père_i \leftarrow \emptyset$
Envoyer $Jeton(0, 0)$ à $j \in Vois_i$

Sur Réception de $Jeton(d, h)$ de j_0

Si $père_i$ est vide
 $père_i \leftarrow j_0$
 $prof_i \leftarrow d + 1$
 $h_i \leftarrow \max(h, prof_i)$

Si $\exists j \in Vois_i, j \neq père_i \wedge j$ non visité

 Envoyer $Jeton(prof_i, h_i)$ à j

Sinon

 Si $père_i \neq \emptyset$
 Envoyer $Jeton(prof_i, h_i)$ à $père_i$

 Sinon

 (h_i) STDP-GLOBAL.